ANALYSIS OF ITERATIVE ALGORITHM

→ Correct algorithms: Correct answer is obtained.

→ Incorrect algorithms: Incorrect answer for valid input instances.

Note: If we control the error rate, incorrect algos are preferred over correct algos (taking more time than incorrect algos).

* Analysis Algorithm:

↳ Many Candidates to single candidates.
↳ Guarantee for all input instances.
↳ Writing program after analysing, solves times.

* Random Access Model:

- One after another, instructions are processed.
- What are the instructions available:
  - Add, Sub, Mul, Div, etc.
  - Data movement, etc.
  - Controls flow: return, if else.

* Pseudo Code:

- return multiple values in single line.

* Linear Search:-

Input : A[1 ---- n] , Value

Output : Index or "Not found"

Best Case
~~Worst Case~~

Linear_Search(A, val):	Cost	#times
for i = 1 to A.length	C_1	1 & n+1
if A[i] == val	C_2	1 & n
return i	C_3	1 & 0
print "Not present"	C_4	0 & 1

Best Case : Value found at location 1.

Worst Case : Value not found.

Best Case :

$T(n) = \sum$ Cost * (#times)

$= C_1(1) + C_2(1) + C_3(1) + C_4(0)$

$T(n) = C_1 + C_2 + C_3$

$\boxed{T(n) = C}$ where $C = C_1 + C_2 + C_3$

Worst Case :

$T(n) = C_1(n+1) + C_2(n) + C_3(0) + C_4(1)$

$= n(C_1 + C_2) + C_1 + C_4$

$\boxed{T(n) = An + B}$ where $A = C_1 + C_2$

$B = C_1 + C_4$

* Sorting Problem :

Input : Array A[1 ... n]
Output : Sorted array

Bubble_Sort (A) : [Ignore Count]

```
Cost  Times (Best Case)
C1    n      for i=1 to (A.length - 1)                    1 to (n-i+1)
                 // count = 0
C2    Σ(n-i+1)   for j=1 to (A.length - i)
C3    Σ(n-i)       if A[j] > A[j+1]
C4    0                A[j] = A[j] + A[j+1]
C5    0                A[j+1] = A[j] - A[j+1]
C6    0                A[j] = A[j] - A[j+1]
                       // count++
                 // if count == 0                          1 + 4
                 // break;
      return A
```

$\frac{n^2+n-2}{2}$

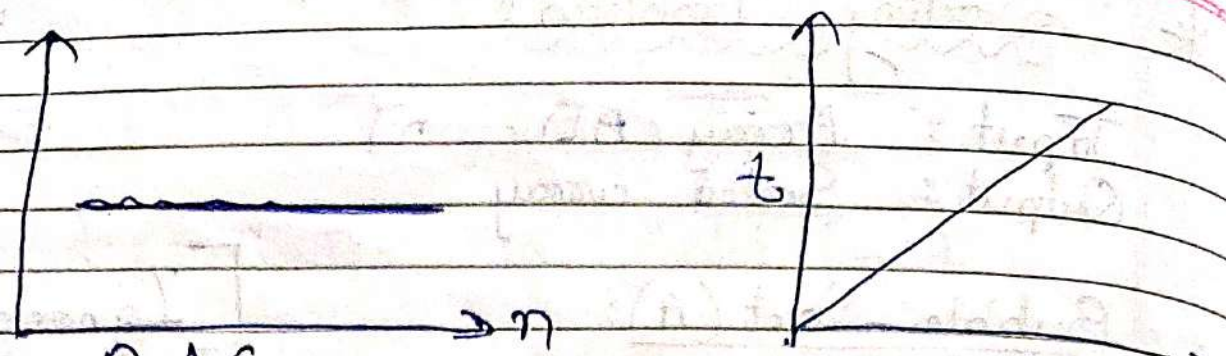
Cost	Times (Best Case)
C1	n
C2	$\sum_{i=1}^{n-1}(n-i+1) = n+(n-1)+\dots+2 = \frac{n(n+1)}{2} - 1 = \frac{(n+2)(n-1)}{2}$
C3	$\sum_{i=1}^{n-1}(n-i) = \frac{(n+2)(n-1)}{2} - (n-1) = \frac{(n-1)n}{2}$
C4	0
C5	0
C6	0

$$T(n) = C_1 n + C_2\left(\frac{(n+2)(n-1)}{2}\right) + C_3\frac{n(n-1)}{2}$$

$$= \frac{2C_1 n + C_2(n^2+n-2) + C_3(n^2-n)}{2}$$

$$= \frac{n^2(C_2+C_3) + n(2C_1+C_2-C_3) - 2C_2}{2}$$

$$T(n) = an^2 + bn + c$$



Linear Search

→ Bubble Sort Worst Case:

Cost	# times
C1	n
C2	$\sum_{i=1}^{n-1} (n-i+1) = \frac{(n+2)(n-1)}{2}$
C3	$\sum_{i=1}^{n-1} (n-i) = n(n-1)/2$
C4	$\sum_{i=1}^{n-1} (n-i)$
C5	"
C6	"

$$T(n) = an^2 + bn + c$$

→ Selection Sort:

```
for i=1 to (A.length - 1)                    Cost
    min = A[i]                               C1
    for j=i+1 to A.length                    C2
        if min > A[j]                        C3
            min = A[j]                       C4
    if min != A[i]                           C5
        temp = min                           C6
        min = A[i]                           C7
        A[i] = temp                          C8
```

→ Best Case: All sorted — Worst Case: Reverse Sorted

Cost	# Times (Best Case)	(Worst Case)
C_1	n	n
C_2	$n-1$	n
C_3	$\sum_{i=1}^{n-1}(n-i+1) = \frac{(n+2)(n-1)}{2}$	$\frac{(n+2)(n-1)}{2}$
C_4	$\sum_{i=1}^{n-1}(n-i) = \frac{n(n-1)}{2}$	$\frac{n(n-1)}{2}$
C_5	0	$\sum_{i=1}^{n-1}(n-i)$
C_6	$n-1$	$n-1$
C_7	0	$n-1$
C_8	0	$n-1$
C_9	0	$n-1$

→ $T(n) = nC_1 + (n-1)C_2 + \frac{(n+2)(n-1)}{2}C_3 + \frac{n(n-1)}{2}C_4 + (n-1)C_6$

Best Case

$= n(C_1+C_2+C_6) - C_2 - C_6 + \frac{(n^2-2+n)C_3 + (n^2-n)C_4}{2}$

$= n^2\left(\frac{C_3}{2}+\frac{C_4}{2}\right) + n\left(C_1+C_2+C_6+\frac{C_3}{2}-\frac{C_4}{2}\right) - C_2 - C_6 - C_3$

$\boxed{T(n) = An^2 + Bn + C}$

→ $T(n) = n^2\left(\frac{C_3}{2}+\frac{C_4}{2}\right) + n\left(C_1+C_2+C_6+\frac{C_3}{2}-\frac{C_4}{2}\right) - C_2 - C_6 - C_3 + \frac{(n^2-n)C_5}{2} + (n-1)C_7 + \dots$

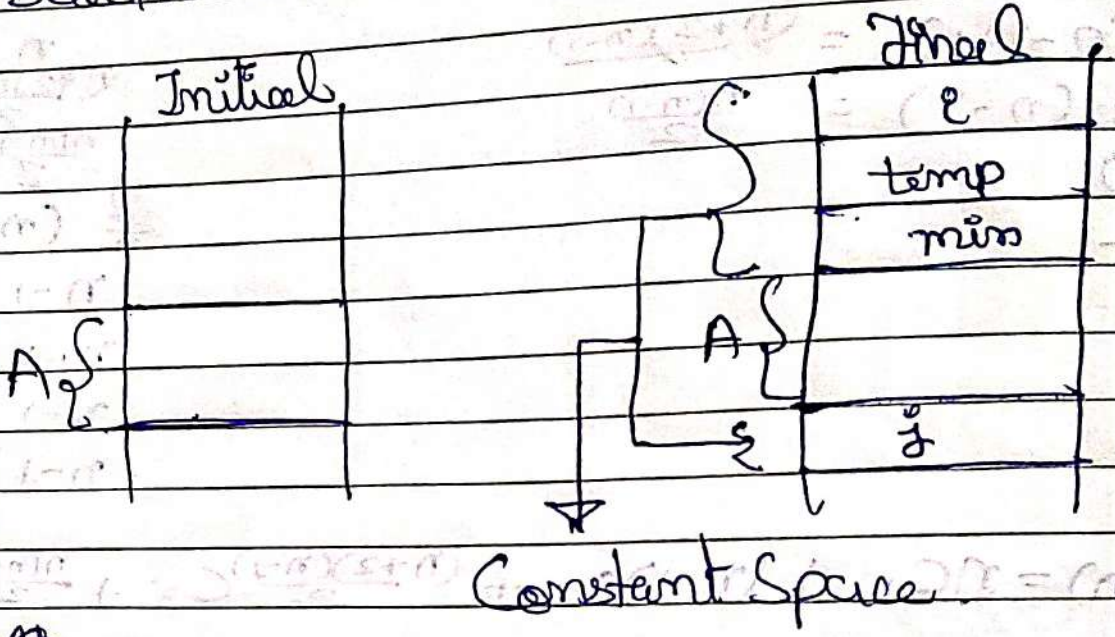
Worst Case

$\boxed{T(n) = an^2 + bn + c}$

5, 6, 5, 4, 5

* In-place Sorting Algorithm:

→ Selection Sort:

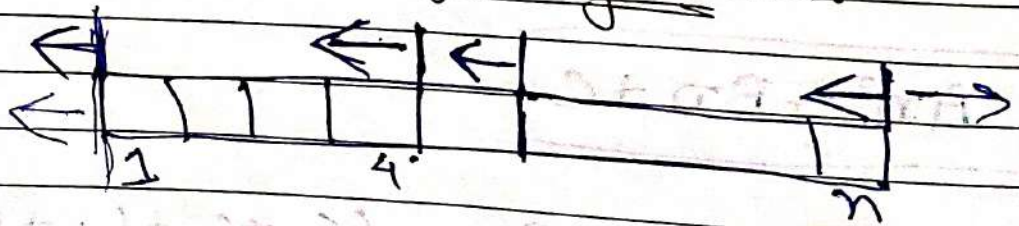


* Stability of Algorithm:

→ Relative order is constant.

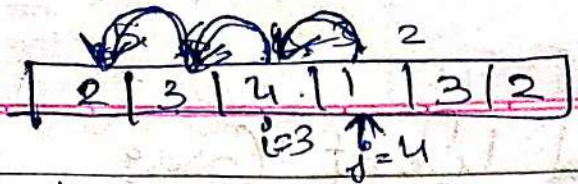
→ Selection Sort is Unstable.

* Correctness of Algorithm:



→ Loop Invariance:
1) Initialization
2) Maintenance
3) Termination

6 5 4 3 2



4 5 6 5 5

Date
Page

Insertion_Sort(A):

```
for   j = 2  to  A.length
      temp = A[j]
      i = j - 1
      while  i > 0  and  A[i] > temp
            A[i+1] = A[i]
            i = i - 1
      A[i+1] = temp
```

Cost	#times (Best Case)	Cost	#times (Reverse sorted (Worst Case))
c_1	n	c_1	n
c_2	$n-1$	c_2	$n-1$
c_3	$n-1$	c_3	$n-1$
c_4	$n-1$	c_4	$\sum_{j=2}^{n} j$
c_5	0	c_5	$\sum_{j=2}^{n} (j-1)$
c_6	0	c_6	$\sum_{j=2}^{n} (j-1)$
c_7	$n-1$	c_7	$n-1$

$T(n) = an + b$

$T(n) = an^2 + bn + c$

Inplace Sort:

* Advantages of Worst Cases:

(1) Upper Ground
(2) Occures frequently
(3) Avg. case is as bad as worst case.

Eg:

N	N^2	N^2+N	$\rightarrow$ % N
10	10^2	110	10%
100	10^4	10100	1%
1000	10^6	1001000	0.1%
10000	10^8	10001000	0.01%

$n \rightarrow \infty$

Note: Hence for large n, we can ignore other terms in time complexity,

$T(n) = an^2 + (bn + c)$

Ignore

Eg:

N	$100N+100$	$2N^2$
1	200	2
10	1100	200
100	10100	20000
1000	100100	2000000

Note: If $n \rightarrow \infty$, constant is not significant. Degree is

2 Growth Of Functions

Date
Page

→ $O(g(n))$: A set of functions.

→ Here, '$=$' symbolises set membership. '$=$' acts like '$\in$'.

∴ $T(n) = O(n^2)$

∴ $T(n) \in O(n^2)$

→ $O(n^2) = T(n)$ ✗

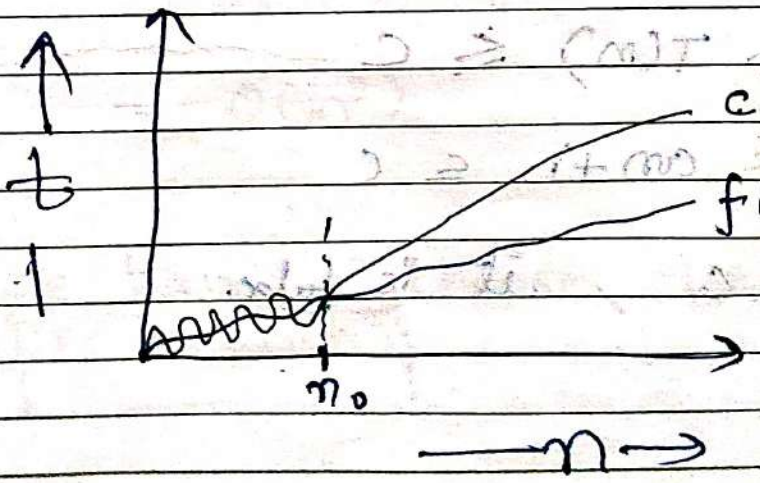
* Asymptotic Notation :

- Theta (Θ)
- Big-oh (O)
- Big-omega (Ω)
- Little-oh (o)
- Little-omega (ω)

↳ $O(g(n))$: { $f(n)$: there exists positive ~~contants~~ constants c & n_0 such that

$$0 \leq f(n) \leq c \cdot g(n)$$

for all $n \geq n_0$



Eg $T(n) = n^2 + 2n$

$T(n) = O(n^2)$ ⇒ Is this true?

So, $0 \leq T(n) \leq C \cdot n^2 \quad \forall n \geq n_0$

$0 \leq n^2 + 2n \leq C \cdot n^2$

If $C = 1$: $0 \leq n^2 + 2n \leq n^2$ Not true.

If $C = 2$: $0 \leq n^2 + 2n \leq 2n^2$

$n_0 = 1$: False

$n_0 = 2$: True

$n_0 = 3$: True

$n_0 = 4$: True

Therefore $\forall n \geq 2$

$C = 2 ; n_0 = 2$

Hence true.

Eg $T(n) = O(n^0)$ or $O(1)$? $T(n) = an + b$

$0 \leq T(n) \leq C$

$0 \leq an + b \leq C$

For all C, it is false.

Eg $T(n) = O(n)$?

$0 \leq T(n) \leq c \cdot n$

$0 \leq an+b \leq cn$

for $c = (a+b)$

It is true for $\forall n \geq n_0$

where $n_0 = 2$ or 3 or ——

Eg $T(n) = O(n^2)$

$\therefore 0 \leq an+b \leq cn^2$

for $c = a+b$

$0 \leq an+b \leq (a+b)n^2$

Always true $\forall n \geq n_0$

where $n_0 = 2$ or 3 or.

→ Linear Search:

$T(n) \neq O(1)$

$= O(n)$

$= O(n^2)$

$= O(n^3)$

⋮

→ But we have to choose closest value.

So, $\boxed{T(n) = O(n)}$

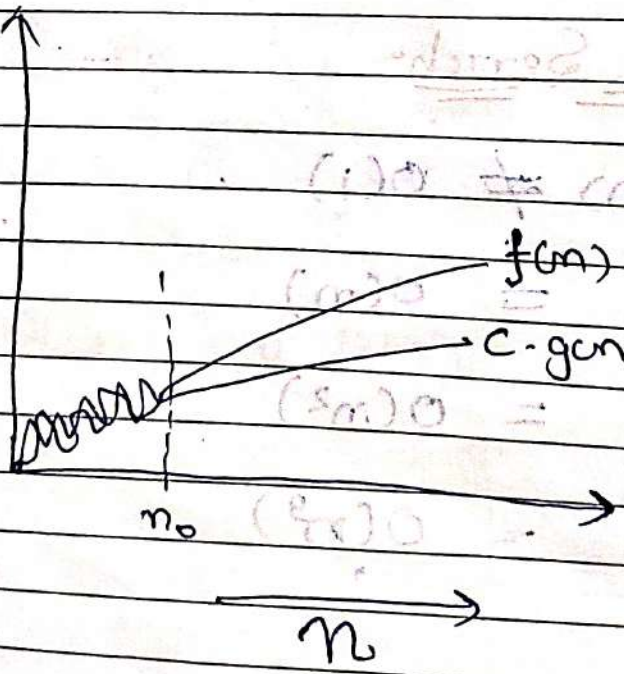
Note: $O(n) = \{c, n, an, an+b, \dots\}$

* $\Omega(n)$:

→ $\Omega(g(n)) = \{ f(n) :$ there exists two constants c & n_0 such that,

$0 \leq c \cdot g(n) \leq f(n)$

for all $n \geq n_0 \}$



→ For linear search: (Worst case)

$T(n) = an + b$

$T(n) = O(n)$

In terms of big omega,

$0 \leq c \cdot n \leq an+b$

True

$T(n) = \Omega(n)$

If $T(n) \leq an+b$
$T(n) = O(n)$

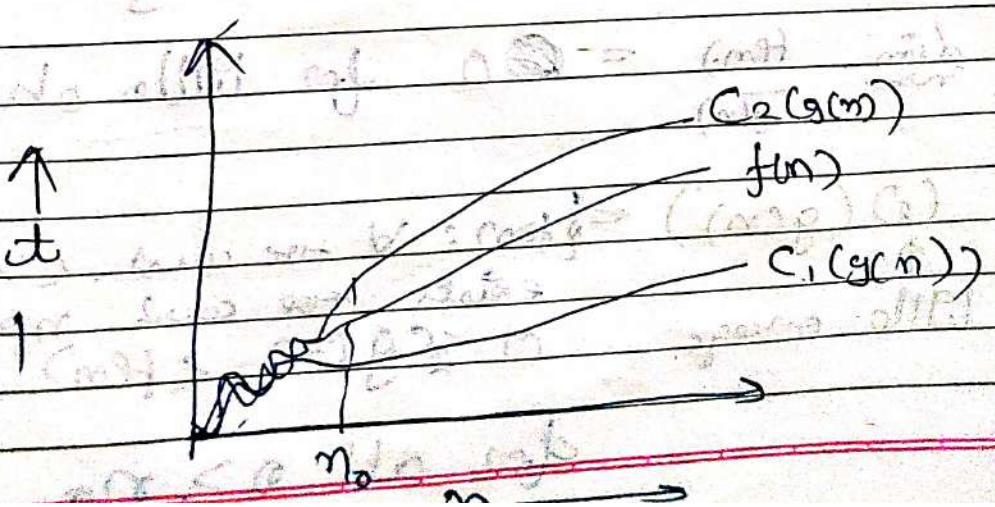
If $T(n) \geq an+b$
$T(n) = \Omega(n)$

If $T(n) = an+b$
$T(n) = \Theta(n)$

* $\Theta(n)$:

$\Theta(g(n))$: $\{ f(n)$: there exists positives const. c_1, c_2 & n_0 such that
$0 \leq c_1 g(n) \leq f(n) \leq c_2 (g(n)$

for all $n \geq n_0$ $\}$



- $\lim_{n \to \infty} \frac{f(n)}{g(n)} = c$, $c > 0$

then $\boxed{f(n) = \Theta(g(n))}$

Eg $f(n) = an + b = 2(1)$; $g(n) = \Theta(1)$
$= O(n^2)$ $= \Theta(n^2)$

For $\Theta(1)$:

$\lim_{n \to \infty} \frac{f(n)}{g(n)} = \infty$ ✗

For $\Theta(n^2)$:

$\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0$ ✗

(A)

* $o(g(n)) = \{ f(n) :$ for any +ve constant $c > 0$, there exists a +ve const. $n_0 > 0$ such that

$0 \leq f(n) < c(g(n))$

for all $n \geq n_0 \}$

↑ Little-oh

- $\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0$ for little oh

* $\omega(g(n)) = \{ f(n) :$ ∀ +ve const. $c > 0$, there exists +ve const. $n_0 > 0$ such that $0 \leq c g(n) < f(n)$

for all $n \geq n_0$

↑ little omega

$\therefore \lim_{n \to \infty} \frac{f(n)}{g(n)} = \infty$ for $c_2(g(n))$

EXAMPLES AFTER FEW PAGES

Eg $T(n) = an^2 + bn + c$

(1) $O(n^2)$:

$0 \le an^2 + bn + c \le c \cdot n^2$

$0 \le a + \frac{b}{n} + \frac{c}{n^2} \le c$

$\boxed{c = a + b + c}$

(2) $\Omega(n^2)$:

$0 \le cn^2 \le an^2 + bn + c$

$0 \le c \le a + \frac{b}{n} + \frac{c}{n^2}$

$\boxed{c \le a}$

(3) $\Theta(n^2)$

$0 \le c_1 \cdot n^2 \le an^2 + bn + c \le c_2 \cdot n^2$

$0 \le c_1 \le a + b/n + c/n^2 \le c_2$

$\boxed{c_1 \le a \ \& \ c_2 \ge (a+b+c)}$

(4) $O(n^3)$ ✓

(5) $\Omega(n^3)$ ✗

(6) $O(n)$ ✗

(7) $\Omega(n)$ ✓

(8) $\Omega(\log n)$ ✓

Eg $f(n) = n$

$g(n) = n^2$

$f(n) = \Theta(g(n))$? ✗

$0 \leq c_1 \cdot n^2 \leq n \leq c_2 \cdot n^2$

$0 \leq c_1 \leq 1/n \leq c_2$

True for c_2 but not for c_1 $c_1 \neq 0$

So, False Ans

Eg $f(n) = 2n^2$

$g(n) = n^2$

$f(n) = o(g(n))$? ✗

$0 \leq 2n^2 < c \cdot n^2$

$0 \leq 2 < c$

false as not true for all $c > 0$

Eg $f(n) = n^2/2$

$g(n) = n^2$

$f(n) = \omega(g(n))$? ✗

$0 \leq c \cdot n^2 \leq n^2/2$

$0 \leq c \leq 1/2$ So false

Eg $f(n) = e^n$

$g(n) = n^2$

$f(n) = \underline{\omega}\ g(n)$

$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \lim_{n\to\infty} \frac{e^n}{n^2} = \lim_{n\to\infty} \frac{e^n}{2n}$

$= \lim_{n\to\infty} \frac{e^n}{2} = \infty$

So, $f(n) = \omega(g(n))$

Eg $f(n) = 3^n$

$g(n) = 2^n$

$f(n) = \underline{\omega}(g(n))$

$\lim_{n\to\infty} \frac{3^n}{2^n} = \infty$

$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$, then $f(n) = o(g(n))$ $\Rightarrow O(g(n))$

$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$, then $f(n) = \omega(g(n))$ $\hookrightarrow \Omega(g(n))$

$\lim_{n\to\infty} \frac{f(n)}{g(n)} = c$, then $f(n) = \Theta(g(n))$

$\hookrightarrow O(g(n))$ & $\Omega(g(n))$

e.g. $f(n) = n^2$
$g(n) = \log n$

$f(n) = \omega\ g(n)$

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \lim_{n\to\infty} \frac{n^2}{\log n} = \frac{2n^2}{\ln 2} = \infty$$

* Exponential functions with different base ___never___ grow at same rate as $n \to \infty$

$$\lim_{n\to\infty} \frac{a^n}{b^n} = \begin{cases} \infty\ ; & a > b \\ 0\ ; & a < b \end{cases}$$

* Logarithmic functions with different bases, grows at ___same___ rate as $n \to \infty$.

$$\lim_{n\to\infty} \frac{\log_a n}{\log_b n} = \frac{\log n}{\log a} \times \frac{\log b}{\log n} = C$$

3 Divide & Conquer

Date
Page

* Recursion: Its a problem solving technique where the solution of problem depends on solutions of smaller instances of same problem.

* (1) Divide the problem
(2) Conquer the subproblems
(3) Combine the solutions

* During recursion, there are 2 cases:
(1) Recursion case
(2) Base case: $\Theta(1)$ time solve the problem.

* Binary Search:

Input: Sorted numbers, element
Output: Index of element or "Not Present"

- Binary_Search (A, val, start, end)

Divide
↳ Divide the problem
$n = n/2 \quad n/2$

Conquer
↳ A[mid] == val
↳ found

start > end
↳ not present

* Binary-Search (A, start, end, val); → T(n)

```
1.  if (start > end)
2.  then print ("Not Found")
3.       |
4.  else                                          Θ(1)
5.       |  mid = (start + end) / 2
6.       |  if (val == A[mid])
7.       |  then print (mid) / return mid
8.       |       |
9.       |  else if (val < A[mid]) } Θ(1)
10.      |       |  then Binary-Search (A, start, mid-1, val)
11.      |       |                      T(n/2)
12.      |  else Binary-Search (A, mid+1, end, val)
                                        T(n/2)
```

→ Recurrence Relation:

$$T(n) = T(n/2) + \Theta(1) \quad \text{---} (1)$$

Back Substitution Method:

$$T(n) = T(n/2) + \Theta(1)$$
$$T(n/2) = T(n/2^2) + \Theta(1)$$
$$T(n/2^2) = T(n/2^3) + \Theta(1)$$
$$\vdots$$
$$T(n/2^{k+1}) = T(n/2^k) + \Theta(1)$$

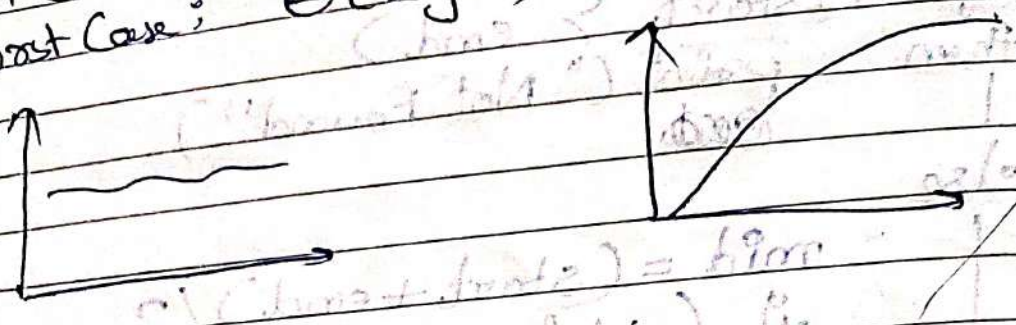
$\therefore \frac{n}{2^k} = 1$

$n = 2^k$

$\boxed{k = \log n}$

$$T(n) = T(n/2^k) + k\Theta(1)$$
$$T(n) = \Theta(1) + k\Theta(1)$$
$$T(n) = \Theta(1) \cdot (\log_2 n + 1)$$
$$\boxed{T(n) = \Theta(\log n)}$$

→ Best Case: $\Theta(1)$
→ Worst Case: $\Theta(\log n)$



* Merge_Sort(A, p, q) → T(n)
  Start end

(Note) Extra space required by Merge Sort: $\Theta(n)$

```
if (p < q)                                   } Θ(1)
    mid = (p+q)/2
    Merge_Sort(A, p, mid)        → T(n/2)
    Merge_Sort(A, mid+1, q)      → T(n/2)
    Merge(A, p, mid, q)          → Θ(n)
```

* Merge(A, p, mid, q):

n1 = mid - p + 1
n2 = q - mid

$T(n) = 2T(n/2) + \Theta(1) + \Theta(n)$

$T(n) = 2T(n/2) + \Theta(n)$; $n > 1$
$T(n) = c$; $n = 1$

Merge (A, p, mid, q):

$n_1 = mid - p + 1$
$n_2 = q - mid$

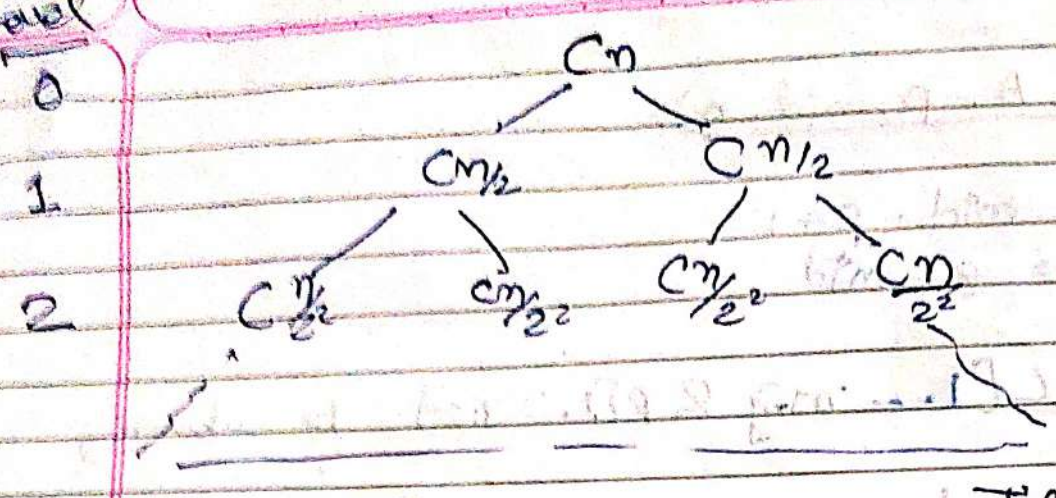
Let $L[1 \ldots n_1+1]$ & $R[1 \ldots n_2+1]$ be subarrays

```
for i = 1 to n1                          }
    L[i] = A[p+i-1]                      }
for j = 1 to n2                          } Θ(n)
    R[j] = A[mid+j+1]                    }
L[n1+1] = ∞; R[n2+1] = ∞;                } C
i = 1                                    }
j = 1                                    }
for k = p to q                           }
    if (L[i] ≤ R[j])                     }
        A[k] = L[i]                      }
        i = i+1                          } Θ(n)
    else                                 }
        A[k] = R[j]                      }
        j = j+1                          }
```

→ Recursion Tree Method:

$T(n)$	cn $T(n/2)$ $T(n/2)$	cn $\frac{cn}{2}$ $\frac{cn}{2}$ $T(\frac{n}{2^2})$ $T(\frac{n}{2^2})$ $T(\frac{n}{2^2})$ $T(\frac{n}{2^2})$
Fig-1	Fig-2	Fig-3

$\frac{n}{2^k} = 1 \Rightarrow n = 2^k \Rightarrow$ $k = \log_2 n$



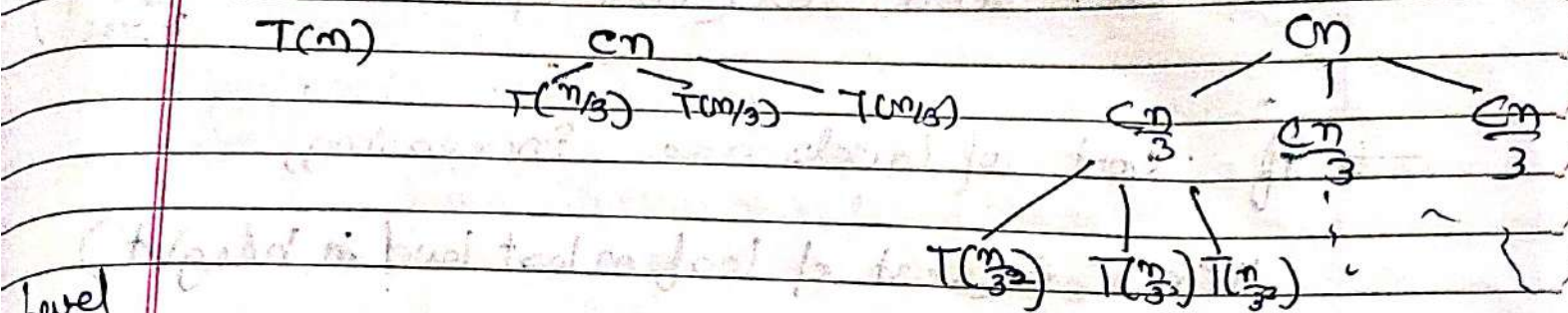
level	#Nodes	Costs
0	1	cn
1	2^1	$2(cn/2) = cn$
2	2^2	$2^2(cn/2^2) = cn$
⋮	⋮	⋮
k	2^k	$2^k \cdot c = cn$

$$T(n) = cn + cn + \cdots + cn$$
$$= (1+k)cn$$
$$= \boxed{cn(\log_2 n + 1)}$$
$$= cn\log n + cn$$

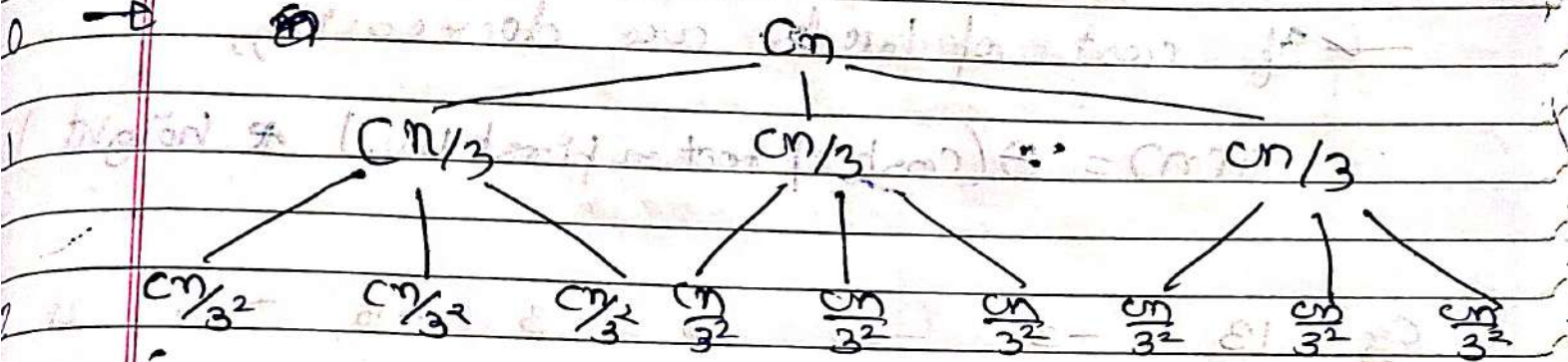
(dominates)

$$\boxed{T(n) = \Theta(n \log n)}$$

Eg1 $T(n) = \begin{cases} 3T(n/3) + cn & ; n \geq 1 \\ c & ; n = 1 \end{cases}$



Level



T(1) — — —

Note:- If cost of levels are same,

$T(n) = \Theta$ (Cost of a level * height of tree)

- If cost of levels are increasing,

$T(n) = \Theta$ (Cost of leaf or last level * height)

- If cost of levels are decreasing,

$T(n) = \Theta$ (Cost of root or first level * height)

Eg 13 −3 −25 20 −3 −16 −23 18

20 −7 12 −5 −22 15 −4 7

Find non-empty contiguous subarray whose elements' sum is maximum.

```
max_sum = 0, begin_index = 0, end_index = 0
for i = 0 to (A.length - 1)
    for j = 1 to A.length
for i = 1 to A.length
    for j = 1 to (A.length - i + 1)
        for k = 1 to i
            sum = 0
            for k = 1 to i
                sum = sum + A[j+k-1]
            if sum > max_sum
                max_sum = sum
                begin_index = j
                end_index = j + i - 1
```

Time Complexity = $\Theta(n^3)$

Better Algorithm:

```
begin_index = 0, end_index = 0, max_sum = 0
for i = 1 to A.length
    for j = 1 to (A.length - i+1)
        sum = sum + A[j+i-1]
        if sum > max_sum
            max_sum = sum
            begin_index = i
            end_index = i + j - 1
```

Time complexity = $\Theta(n^2)$

* Divide & Conquer : (Same problem)

2 | -3 | 3 | 2 | 4 | -2 | 1

Maximum Sum Subarray

MSS(A, start, end);

```
mid = (start + end) / 2
a = mssLeft(A, start, mid)
b = mssRight(A, mid+1, end)
c = mssBoth(A, mid)
```

→ Find-max-subarray (A, low, high):

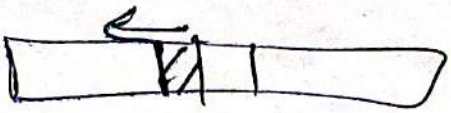
if low = high
    return (low, high, A[low])
else
    mid = (low + high)/2
    (low-left, high-left, sum-left) = Find-max-subarray (A, low, mid)
    (low-right, high-right, sum-right) = Find-max-subarray (A, mid+1, high)
    (low-both, high-both, sum-both) = Both-side (A, low, mid, high)
    return max (sum-1, sum-2, sum-3)

if sum-left ≥ sum-right and sum-left ≥ sum-both
    return (low-left, high-left, sum-left)
else if sum-right ≥ sum-left and sum-right ≥ sum-both
    return (low-right, high-right, sum-right)
else
    return (low-both, high-both, sum-both)

→ Both-side (A, low, mid, high):

max-sum = A[mid]
sum = A[mid]
begin-index = mid
end-index = mid

} O(1)



Date
Page

```
for i = mid-1 downto low
    sum = sum + A[i]
    if sum > ~~mid-sum~~ max-sum
        max-sum = sum
        begin-index = i
        end-index = mid
```

$\Theta(n)$

```
sum = max-sum
for i = mid+1 to high
    sum = sum + A[i]
    if sum > max-sum
        max-sum = sum
        ~~begin-index~~
        end-index = i
```

$\Theta(n)$

And-max-subarray:

$T(n) = 2T(n/2) + \Theta(n)$

$$T(n) = \Theta(n \log n)$$

Goal: Find multiplication of two integers.

Multiply (a[1...p], b[1...q], base) :

// rightmost digit at index 1

2 3 5 7

a[4] b[3] a[2] a[1]

x = a // x shows result array

for i = 1 to q // i represents array b

carry = 0

for j = 1 to p // j shows array a

pdt = a[j] * b[i]

res[x] = pdt + carry

res[x] = pdt + carry

carry = pdt % base

for i = 1 to q

carry = 0

for j = 1 to p

product[i+j-1] += carry + a[j] * b[

carry = product[i+j-1] / base

product[i+j-1] %= base

product[i+p] += carry

return product

$T(n) = \Theta(n^2)$

6 6

```
  23
 x32
 ---
  46
 690
 ---
 736
```

Date
Page

→ Can we improve the performance?

$T(n) < n^2$

→ Divide & Conquer Approach:

a: 2 3 4 5

b: x 3 2 1 6

(1) Divide a & b into two parts

a_L a_R

2 3 4 5

b_L b_R

x 3 2 1 6

(2) Conquer: Recursively solve subproblems.

$2345 = 23 \times 10^2 + 45$

$1234 = 12 \times 10^2 + 34$

$\therefore a \cdot b = (23 \times 10^2 + 45) \cdot (12 \times 10^2 + 34)$

$= 23 \cdot 12 \cdot 10^4 + (a_L \cdot b_R + a_R \cdot b_L) 10^2$

$(2 \cdot 34 + 12 \cdot 45) 10^2$

$+ 34 \cdot 45$

$= \underbrace{a_L \cdot b_L}_{①} (base)^n + (\underbrace{a_L \cdot b_R}_{②} + \underbrace{a_R \cdot b_L}_{③}) (base)^{n/2}$

$+ \underbrace{a_R \cdot b_R}_{④}$

MulProblem (a, b, n)

* Mulproblem $(a / 10^{n/2}, b / 10^{n/2})$
  Mulproblem $(a \% 10^{n/2}, b \% 10^{n/2})$

T(n) Mulproblem (a, b):

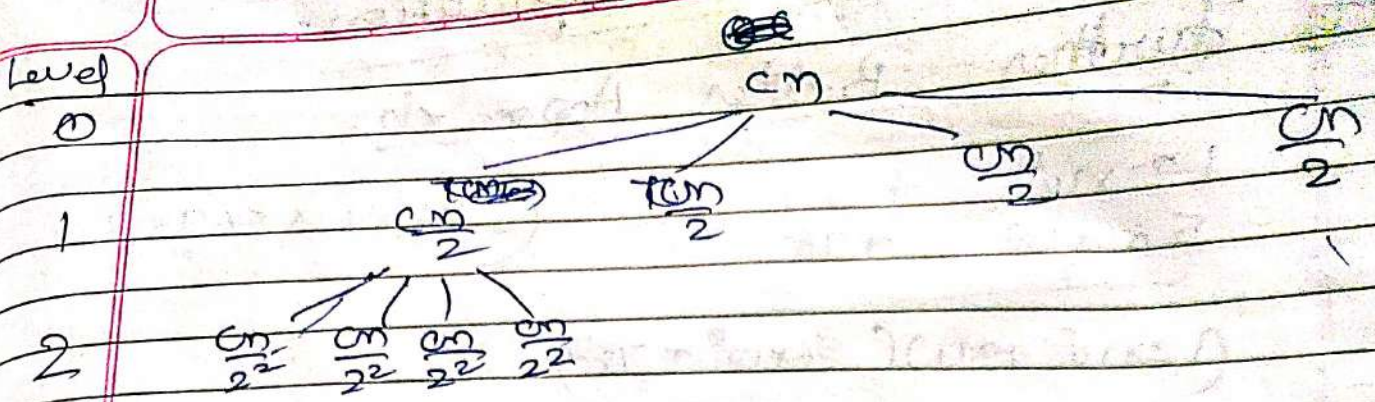
$\Theta(1)$
- n = max (size of a, size of b)
- if $n == 1$
  - return
- a_L =
- b_L =
- a_R =
- b_R =

$T(n/2)$ $P1$ = Mulproblem (a_L, b_L)
$T(n/2)$ $P2$ = Mulproblem (a_L, b_R)
$T(n/2)$ $P3$ = Mulproblem (a_R, b_L)
$T(n/2)$ $P4$ = Mulproblem (a_R, b_R)

$\Theta(n)$ $P = P1 \cdot base^{n} + (P2 + P3)\, base^{n/2} + P4$
$\Theta(1)$ return P

Time complexity:

$T(n) = 4T(n/2) + \Theta(n)$



k T(1) T(1) ——— T(1)

$\frac{n}{2^k} = 1 \Rightarrow n = 2^k \Rightarrow \boxed{k = \log_2 n}$

Level	Node	Sum
0	4^0	cn
1	4^1	2cn
2	4^2	2^2 cn
⋮	⋮	⋮
k	4^k	$2^k \cdot cn$

$T(n) = cn(2^0 + 2^1 + 2^2 + \dots 2^k)$

$= cn((1)(2^k - 1))$

$= cn(n-1)$

$= cn^2 - cn$

$\boxed{T(n) = cn^2}$

$\boxed{T(n) = \Theta(n^2)}$

Karatsuba Multiplication:-

Further Better Approach:

$12 \times 10^2 + 34$
$56 \times 10^2 + 78$

(1234×5678)

$(12\times10^2+34)(56\times10^2+78)$

$= \underbrace{12\cdot56\cdot10^4}_{①} + \underbrace{(12\cdot78+34\cdot56)10^2}_{②\ a_L b_R \quad ③\ a_R b_L} + \underbrace{34\cdot78}_{④\ a_R b_R}$

$a_L \cdot b_L$

$(a_L + a_R)(b_L + b_R)$
$(12+34)(56+78)$

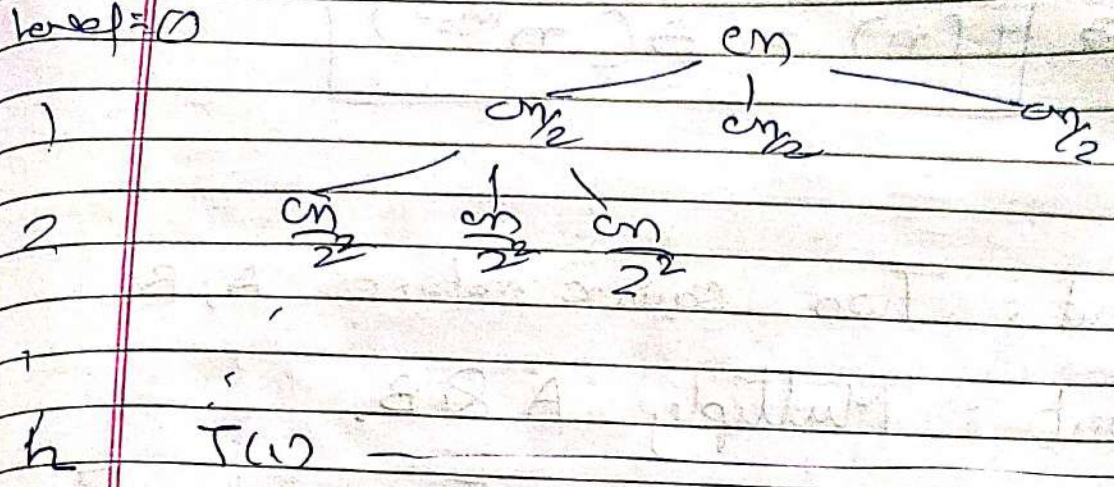
$= a_L b_L + a_R b_R + a_L b_R + a_R b_L$

$a_L b_R + a_R b_L = \underbrace{(a_L + a_R)\cdot(b_L + b_R)}_{①} - \underbrace{a_L b_L}_{②} - \underbrace{a_R b_R}_{③}$

So only 3 multiplication needed.

```
Karatsuba Mulproblem (a,b)-
O(1) {  n = max(size of a, size of b)
        if n <= 1
            return a.b
     P1 = Mulproblem (aL, bL) ---- T(n/2)
     P2 = Mulproblem (aR, bR) ---- T(n/2)
     P3 = Mulproblem (aL+aR, bL+bR) ---- T(n/2)
O(n) { P = P1 . base^n + (P3-P1-P2) . base^(n/2) + P2
O(1) { return P
```

$T(n) = 3T(n/2) + \Theta(n)$



→ e

level	Nodes	Sum
0	1	$cn.(1)$
1	3^1	$3(cn/2)$
2	3^2	$3^2(cn/2^2)$
⋮	⋮	⋮
h	3^h	$3^h(cn/2^h)$

→ e $\frac{n}{2^h} = 1$ → e $\boxed{n = 2^h}$ → e $\boxed{h = \log_2 n}$

↳ ❀ $T(n) = cn\left(1 + \frac{3}{2} + \left(\frac{3}{2}\right)^1 + \cdots + \left(\frac{3}{2}\right)^h\right)$

$= cn\left(\frac{(1)\,(3/2)^h - 1}{1/2}\right)$

$= 2cn\left(\frac{3^{\log_2 n}}{n} - 1\right)$

$= 2cn\left(n^{\log_2 3 - 1} - 1\right)$

$= 2c\, n^{\log_2 3} - 2cn$

$$T(n) = \Theta(n^{\log_2 3})$$

4

Input : Two square matrixes A, B

Output : Multiply A & B.

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \quad B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}$$

$$AB = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{22} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{bmatrix}$$

$= 2 \times 2 \& 2$

$= 8$

if $n \times n$

then $T(n) = \Theta(n^3)$

SMM (A, B) ;

n = size of A

for i = 1 to n

for j = 1 to n

$C_{ij} = 0$

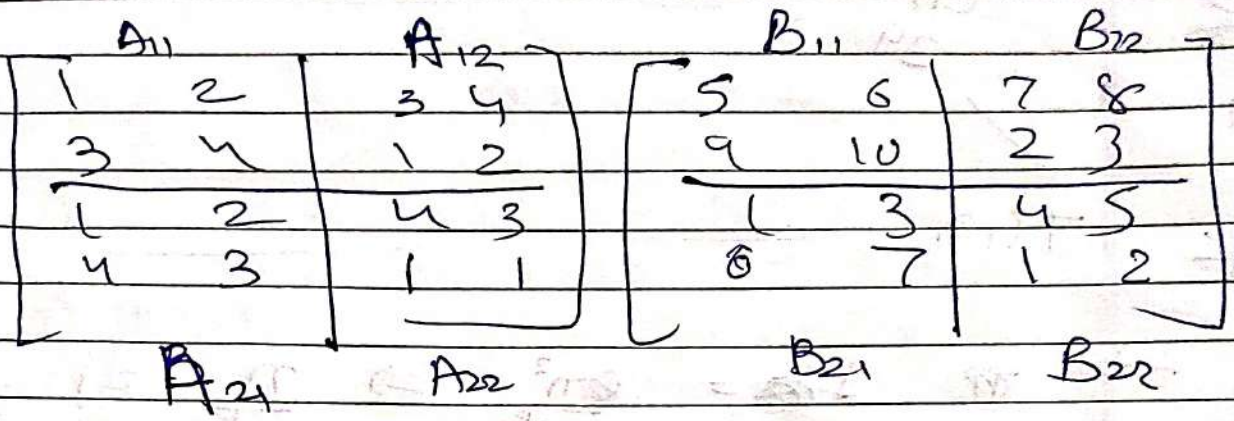
for k = 1 to n

$c_{ij} = c_{ij} + a_{ik} \cdot b_{kj}$

return C

$T(n) = \Theta(n^3)$

Divide & Conquer Approach:



SMM(A, B): [a, b are integers] → T(n)

n = A.rows
Let C be a new n×n matrix
if n == 1
    $c_{11} = a_{11} \cdot b_{11}$
} Θ(1)

else

$C_{11} = SMM(A_{11}, B_{11}) + SMM(A_{12}, B_{21})$
(Matrix Add)

$C_{12} = SMM(A_{11}, B_{12}) + SMM(A_{12} + B_{22})$

$C_{21} = SMM(A_{21}, B_{11}) + SMM(A_{22}, B_{21})$

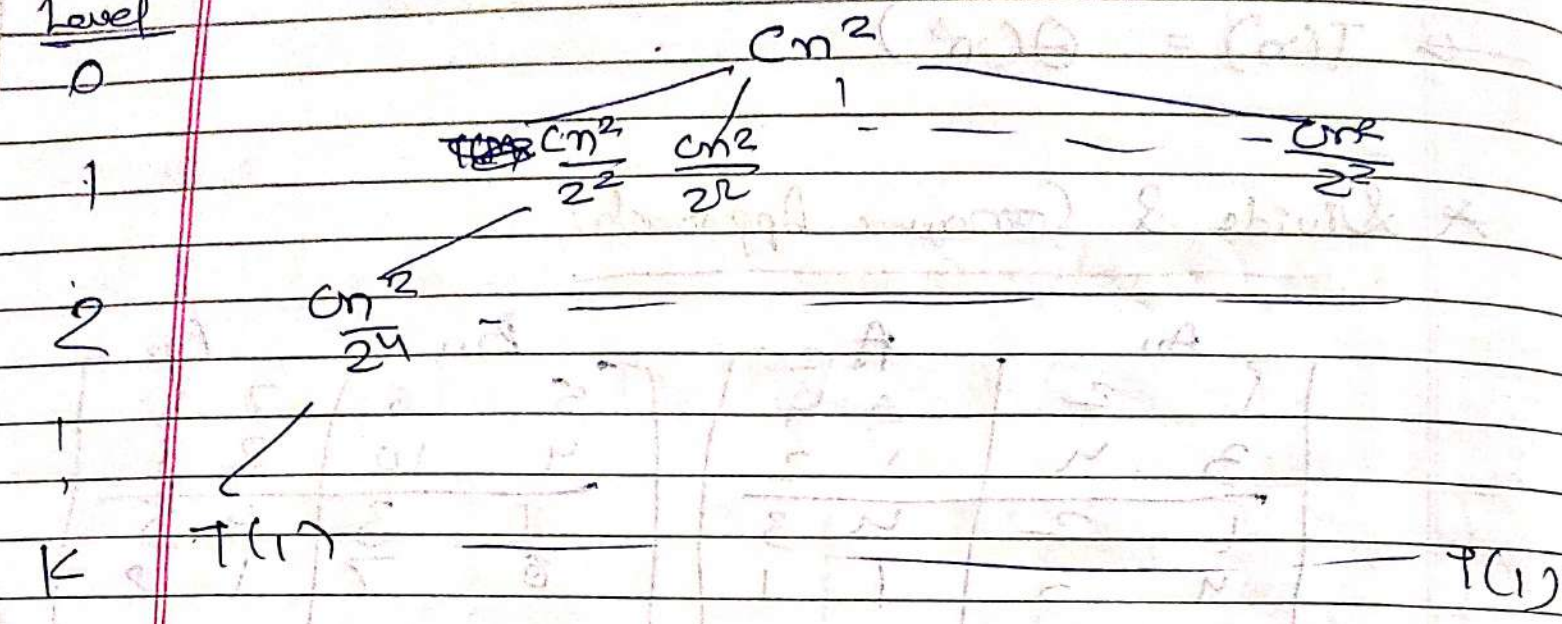
$C_{22} = SMM(A_{21}, B_{12}) + SMM(A_{22}, B_{22})$

8T(n/2) + Θ(n²)

return C

Date
Page

— e $T(n) = 8\,T(n/2) + \Theta(n^2)$



→ $\frac{n^2}{2^{2k}} = 1 \Rightarrow \frac{n}{2^k} = 1$

$n = 2^k$

$\boxed{k = \log_2 n}$

— e $T(n) = cn^2 + 8\left(\frac{cn^2}{2^2}\right) + 8^2\left(\frac{cn^2}{2^4}\right) + \cdots + k \cdot 8^k(c)$

$= cn^2 + 2 \cdot cn^2 + 2^2 \cdot cn^2 + \cdots + k \cdot cn^3$

$T(n) = \Theta(n^3)$

* The Master Method:

↳ $T(n) = a \cdot T(n/b) + f(n)$

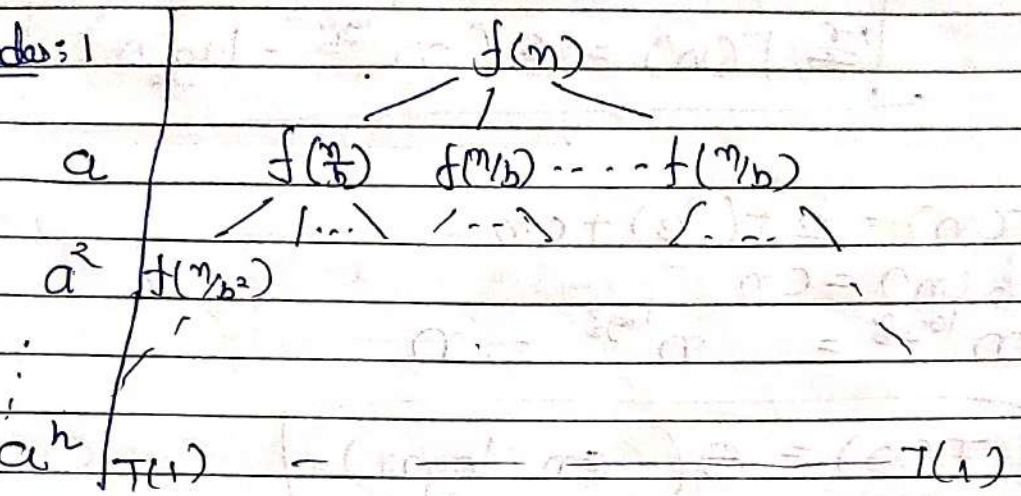
$n \equiv$ input size

$a \equiv$ No. of subproblems

$n/b \equiv$ size of subproblem

$f(n) \equiv$ Cost of Divide + Combine + other subproblems

↳ Nodes: 1



→ $\frac{n}{b^h} = 1 \Rightarrow n = b^h \Rightarrow h = \log_b n$

→ $T(n) = f(n) + af(n/b) + a^2 f(n/b^2) + \cdots + a^h T\left(\frac{n}{b^h}\right)$

→ $a^h f\left(n/b^h\right) = a^h T(1) = a^{\log_b n} \cdot T(1)$

$= cn^{\log_b a}$

Case 1: $f(n) > n^{\log_b a}$ [Polynomially larger (degree)]

So, $f(n)$ will be largest then

$T(n) = \Theta(f(n))$

$T(n) = \Theta(f(n))$

Case 2: $f(n) < n^{\log_b a}$

↳ $T(n) = \Theta(n^{\log_b a})$

Case 3: $f(n) = n^{\log_b a}$

↳ $T(n) = \Theta(f(n) \cdot \log_b n)$

or $T(n) = \Theta(n^{\log_b a} \cdot \log_b n)$

Eg: $T(n) = 2T(n/2) + cn$

$f(n) = cn$

$n^{\log_b a} = n^{\log_2 2} = n$

↳ $T(n) = \Theta(cn \cdot \log n)$

Eg: $T(n) = 8T(n/2) + n^2$

$f(n) = n^2$

$n^{\log_b a} = n^3$

↳ $T(n) = \Theta(n^3)$

Case 1: If $f(n) = O\left(n^{\log_b a - \varepsilon}\right)$; $\varepsilon > 0$

then $f(n)$ is polynomially smaller

then $T(n) = \Theta(n^{\log_b a})$

Date
Page

Case 2: If $f(n) = \Theta(n^{\log_b a})$, then

$$T(n) = \Theta(n^{\log_b a} \cdot \log n)$$

Case 3: If $f(n) = \Omega(n^{\log_b a + \varepsilon})$; $\varepsilon > 0$

and if $a \cdot f(n/b) \leq c f(n)$ for some $c < 1$ (Regularity condition)

$\forall n \geq n_0$.

then

$$\boxed{T(n) = \Theta(f(n))}$$

Eg $T(n) = 9 \cdot T(n/3) + n \Rightarrow \boxed{T(n) = \Theta(n^2)}$

Eg $T(n) = 2\,T(n/4) + n$

$f(n) = n$

$n^{\log_b a} = n^{1/2}$

i.e $a \cdot f(n/b) \leq c f(n)$ $\exists\, c < 1$

$2 \cdot \left(\frac{n}{4}\right) \leq cn$

$n \leq 2cn$

$1 \leq 2c$

$\boxed{c \geq \frac{1}{2}}$

i.e so regularity is satisfied.

i.e So by case 3,

$$\boxed{T(n) = \Theta(n)}$$

Eg $T(n) = 4 \cdot T(n/2) + n\log n$

$\Rightarrow f(n) = n\log n$

$n^{\log_b a} = n^{\log_3 4}$

↳ Is $f(n)$ asymptotically smaller than $n^{\log_b a}$?

$$\lim_{n\to\infty} \frac{f(n)}{n^{\log_b a}} = \lim_{n\to\infty} \frac{n\log n}{n^{1+x}} = \lim_{n\to\infty} \frac{\log n}{n^x}$$

$$= \lim_{n\to\infty} \frac{1/n}{x\,n^{x-1}} = \lim_{n\to\infty} \frac{1}{x\,n^x} \simeq \boxed{0}$$

↳ So, $f(n) = O(n^{\log_b a})$

↳ Is $f(n)$ polynomially smaller than $n^{\log_b a}$?

$$\lim_{n\to\infty} \frac{f(n)}{n^{\log_b a - \varepsilon}} = \lim_{n\to\infty} \frac{n\log n}{n^{1+x-\varepsilon}} = \lim_{n\to\infty} \frac{\log n}{n^{x-\varepsilon}}$$

$$= \lim_{n\to\infty} \frac{1/n}{(x-\varepsilon)\,n^{x-\varepsilon-1}} = \boxed{0}$$

↳ So, $f(n) = O(n^{\log_b a - \varepsilon})$

↳ So, By Case 1,

$$\boxed{T(n) = \Theta(n^{\log_3 4})}$$

Eg $T(n) = 2T(n/2) + n\log n$

n $\quad$ $n\log n$

↳ Asymptotically:

$$\lim_{n\to\infty} \frac{n\log n}{n} = \infty$$

↳ $f(n) = \Omega(n^{\log_b a})$

↳ Polynomially:

$$\lim_{n\to\infty} \frac{n\log n}{n^{1+\varepsilon}} = \lim_{n\to\infty} \frac{\log n}{n^{\varepsilon}} = \lim_{n\to\infty} \frac{1}{\varepsilon n^{\varepsilon}}$$

$= 0$

$f(n) = O(n^{\log_b a + \varepsilon})$

↳ Hence, $f(n)$ is asymptotically larger but not polynomially.
So Master theorem can't be applied.

Note:
1. $f(n)$ can't be -ve.
2. a must be ≥ 1
3. a must be a constant (2^n - not true)

Eg $T(n) = 4T(n/2) + \frac{n^2}{\log n}$

- $\lim_{n\to\infty} \frac{n^2}{\log n \cdot n^2} = 0$
- $\lim_{n\to\infty}$

Note: If $f(n) = \Theta(n^{\log_b a} \cdot (\log^k n))$ where $K \geq 0$

then, $\boxed{T(n) = \Theta(n^{\log_b a} \cdot (\log n)^{K+1})}$

* ~~Back~~ Substitution Method:

(1) Guess the Solution
(2) Use Math. Induction to find the constants & validate the guessed solution.

Eg
$$T(n) = \begin{cases} 0 & ; n = 0 \\ 2 \cdot T(n-1) + 1 & ; n > 0 \end{cases}$$

Sol
$T(1) = 2T(0) + 1 = 1$
$T(2) = 2T(1) + 1 = 3$
$T(3) = 7$
$T(4) = 15$
$T(5) = 31$

$T(n) = 2^{n-1}$

① Base Case : (That we did)
(small values check the soln)

② Induction hypothesis : $\{1, \ldots, n-1\}$

③ Inductive Step : Prove $T(n)$.

→ Ind. Hp : Assume that guessed soln is correct $\forall n \in \{1, 2, \ldots, n-1\}$

→ I. Step : Prove $T(n) = 2^n - 1$

$$T(n) = 2T(n-1) + 1$$
$$= 2 \cdot (2^{n-1} - 1) + 1$$
$$= 2^n - 2 + 1$$
$$= 2^n - 1$$

Hence proved.

Q Show that soln of
$$T(n) = 2T(\lfloor n/2 \rfloor) + n$$
is $\Theta(n \log n)$.

Sol Base Case:

$T(1) = 1 \leq C \cdot 1 \log 1 = 0$? False
$T(2) = 2T(1) + 2 = 4$
$T(3) = 2T(1) + 2 = 4$
$T(4) = 2T(2) + 2 = 10$
$T(5) = 2T(2) + 2 = 10$
$T(2) = 4 \leq C \cdot 2 \log 2$ True for $C \geq 2$
$T(3) = 4 \leq C \cdot 3 \log 3$ True for $C \geq 2$
$T(4) = 10 \leq C \cdot 4 \log 4$ True "
$T(5) = 10 \leq C \cdot 5 \log 5$ True "

So, $\forall C \geq 2$, $n_0 = 2$

Date
Page

I.H.: Assume it is true for $n \in \{2, 3, \ldots, n-1\}$

$\Rightarrow T(\lfloor n/2 \rfloor) \leq c \cdot \lfloor n/2 \rfloor \cdot \log(\lfloor n/2 \rfloor)$

I. Step: $T(n) = 2T(\lfloor n/2 \rfloor) + n$

$T(n) \leq 2 \cdot c \cdot \lfloor \frac{n}{2} \rfloor \cdot \log(\lfloor \frac{n}{2} \rfloor) + n$

$T(n) \leq 2 \cdot c \cdot \left(\frac{n}{2}\right) \log\left(\frac{n}{2}\right) + n$

$\left[\because \lfloor n/2 \rfloor \leq n/2\right]$

$T(n) \leq cn(\log n - \log 2) + n$

$T(n) \leq cn\log n - cn + n$

$T(n) \leq cn\log n + n(1 - c)$

So $1 - c \leq 0$

$c \geq 1$

And we have $c \geq 2$ & $n_0 = 2$

So yes it is true.

$T(n) \leq cn\log n$

So, $\boxed{T(n) = O(n\log n)}$

Eg. Show that soln of

$T(n) = 3 \cdot T(\lfloor n/4 \rfloor) + n^2$ is $O(n^2)$

Sol Base Case : $T(1) = 1 \leq C(1)^2$? True for $C \geq 1$

$T(2) = 4 \leq 4C$? True for $C \geq 1$

$T(4) = 3+16 \leq 16C$? True for $C \geq 2$

$T(6) = 28 \leq 25C$? True for $C \geq 2$

$T(8) = 64+12 \leq 64C$? True for $C \geq 2$

I.H : Assume $T(m) = O(m^2)$

$1 \leq m < n$ & $1 \leq \lfloor \frac{m}{4} \rfloor < n$

$m \geq 4$

I. Step : $T(n) = 3T(\lfloor n/4 \rfloor) + n^2$

$\leq 3C \cdot \left(\lfloor \frac{n}{4} \rfloor\right)^2 + n^2$

$\leq \frac{3}{16} Cn^2 + n^2$

$\leq \left(\frac{3}{16}C + 1\right) n^2$

So, $T(n) = O(n^2)$

$O(n)$

$\leq cn$

But To prove $T(n) \leq cn^2$

$$\left(\frac{3c}{16} + 1\right) n^2 \leq cn^2$$

$$1 \leq \frac{13}{16} c$$

$$\boxed{c \geq \frac{16}{13}}$$

$T(n) \leq cn^2$ iff $\boxed{c \geq \frac{16}{13}}$

$\boxed{n_0 = 1}$

Eg Show that soln of $T(n) = T(n/3) + T(2n/3) + O(n)$ is $O(n \log n)$.

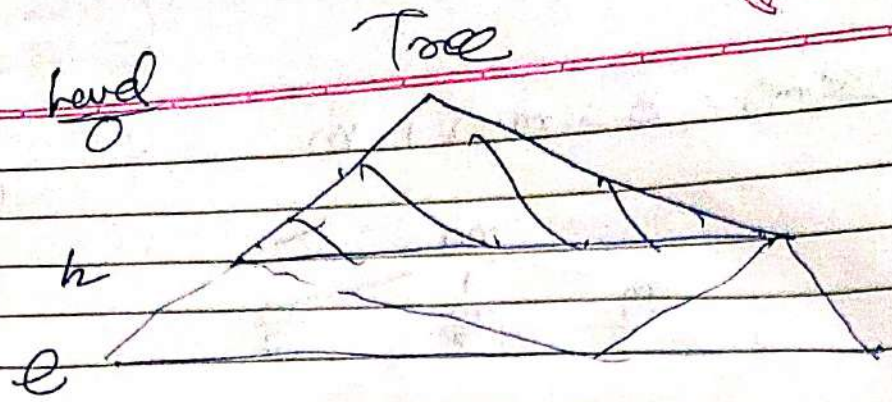
Sol Base Case: $T(1) \leq c \cdot (1)$

$T(2) \leq c + 2c = 3c$

$T(3) \leq 7c$

$T(4) \leq 8c$

$T(5) \leq 13c$



→ $\frac{n}{3^h} = 1$ ← e So upto level h : $[T(n)]_{min} = n \log_3 n$

← e $\frac{2n}{3^e} = 1 \Rightarrow$ So atleast $[T(n)]_{max} = n \log_{3/2} n$

↳ So, $T(n) = O(n \log n)$ [∵ Best Guess (if not given)]

✱ Changing Variables:

(1) $T(n) = 2\,T(\sqrt{n}) + \log n$

↳ $n = 2^m \Rightarrow m = \log_2 n$

↳ $T(2^m) = 2 \cdot T(2^{m/2}) + m$

Let $T(2^m) = S(m)$

$S(m) = 2 \cdot S(m/2) + m$

$S(m) = \Theta(m \log m)$

$\therefore T(n) = \Theta(m \log m)$

$$\boxed{T(n) = \Theta(\log n \cdot \log\log n)}$$

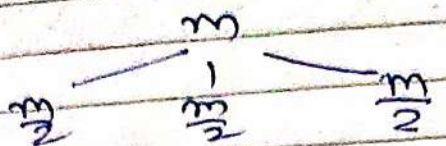
(2) $T(n) = 3 \cdot T(\sqrt{n}) + \log n$

$n = 2^m \Rightarrow m = \log_2 n$

↳ $T(2^m) = 3 \cdot T(2^{m/2}) + m$

Let $T(2^m) = S(m)$

↳ $S(n) = 3\,S(n/2) + n$



Level	Nodes
0	1
1	3^1
.	.
.	.
k	3^k

$\text{Sum} = 3^k \cdot \frac{n}{2^k} = 1$

$= 3^{\log_2 n} \cdot \frac{n}{2^k} = 1$

$2^k = n$

$S(n) = n^{\log_2 3}$

↳ $\boxed{T(n) = \Theta(n^{\log_2 3})}$

(3) $T(n) = \begin{cases} 2 \cdot T(\sqrt{n}) + 1 & ; n > 2 \\ 2 & ; n \le 2 \end{cases}$

$n = 2^m \;\therefore\; m = \log_2 n$

↳ $T(2^m) = \begin{cases} 2 \cdot T(2^{m/2}) + 1 & ; 2^m > 2 \Rightarrow m > 1 \\ 2 & ; 2^m \le 2 \Rightarrow m \le 1 \end{cases}$

↳ $S(m) = \begin{cases} 2 \cdot S(m/2) + 1 \\ 2 \end{cases}$

$S(m) = \Theta(m)$

$\boxed{T(n) = \Theta(\log n)}$

(4) $T(n) = \sqrt{n} \cdot T(\sqrt{n}) + n$

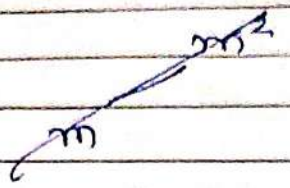
↳ $n = 2^m$

$m = \log_2 n$

$T(2^m) = 2^{m/2} \cdot T(2^{m/2}) + 2^m$

Let $m = m^2$

$T(m^2) = m \cdot T(m) + m^2$



$S(m) = 2^{m/2} \cdot S(m/2) + 2^m$

$= 2^{m/2}\left[2^{m/2^2} \cdot S\left(\frac{m}{2^2}\right) + 2^{m/2}\right] + 2^m$

$= 2^{\frac{m}{2}+\frac{m}{2^2}} \cdot S\left(\frac{m}{2^2}\right) + 2^m + 2^m$

$= 2^{\frac{m}{2}+\frac{m}{2^2}+\frac{m}{2^3}} \cdot S\left(\frac{m}{2^3}\right) + 3(2^m)$

$\frac{m}{2}+\frac{m}{2^2}+\cdots+\frac{m}{2^k}$

$= 2 \cdot S\left(\frac{m}{2^k}\right) + k(2^m)$

$= 2^{m\left[1-\frac{1}{2^k}\right]} \cdot S\left(\frac{m}{2^k}\right) + k \cdot 2^m$

$= 2^{m-1} \cdot S(1) + (\log m) \cdot 2^m$

dominate

So, $S(m) = \Theta(2^m \log m)$

$T(n) = \Theta(2^{\log_2 n} \cdot \log\log n)$

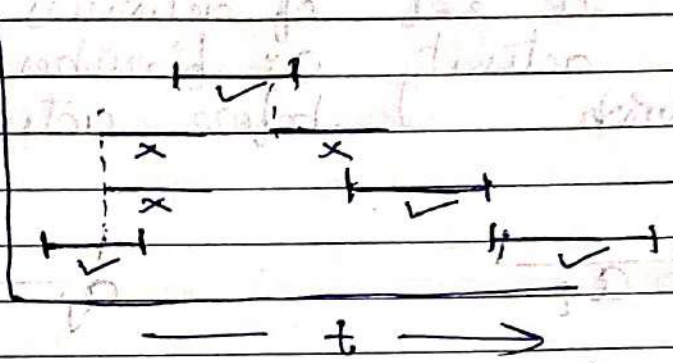
$$\boxed{T(n) = \Theta(n \cdot \log\log n)}$$

Greedy Algorithm

* Activity Selection Problem:

→ $A_1, A_2, \ldots A_n$ [Resources]

→ Output: Max. sized subset who are mutually compatible.



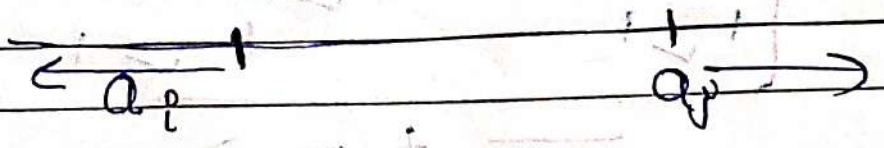
→ Goal:

	Start Time	finish time
a_1	3:00	4:00
a_2	3:30	5:00
a_3	3:45	6:00
a_4	5:00	6:00

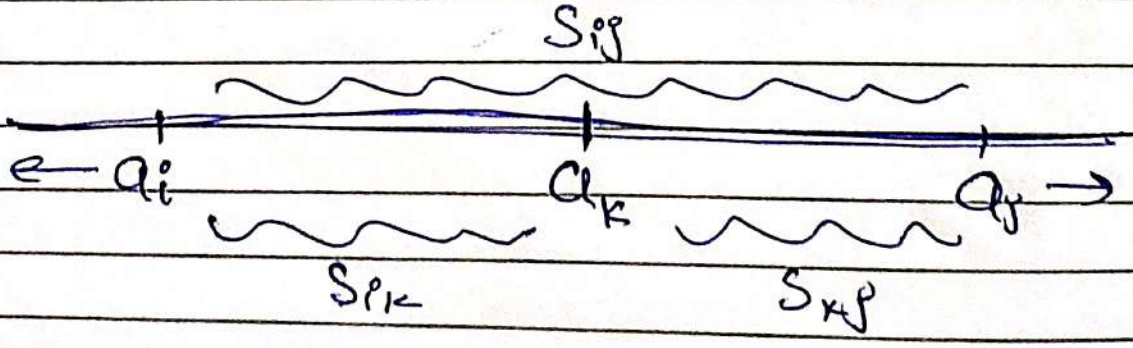
* Properties of DP :

① Overlapping subproblems
② Optimal substructure
↳ Optimal sol^n of original problem incorporates optimal sol^n of subproblems

→ Let S_{ij} be set of activities that start after activity a_i finishes and finish before activity a_j starts.



→ A_{ij} - Max. size subset of mutually compatible activities from S_{ij}



$$A_{ij} = A_{ik} + A_k + A_{kj}$$

↳ Let $c[i,j]$ be size of optimal sol^n for S_{ij}.

$$c[i,j] = \max_{a_k \in S_{ij}} [c[i,k] + c[k,j] + 1]$$

* Greedy Choice:

↳ Input: S_{ij}, Resource [R]

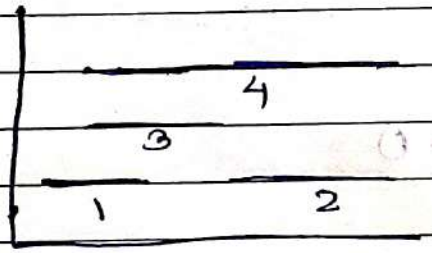
Sorted based on finish time

$f_i \leq f_{i+1} \leq f_{i+2}$

↳ Choose an activity that frees the resource as early as possible.

* How to prove that greedy choice leads us to optimal soln?

Activity selected by greedy choice will be in one of the max. size subset of mutually compatible activities



$A_{ij} = \{a_1, a_2\} \subseteq \{a_3, a_4\}$

Note: Read Textbook

If we choose a_1 then $a_1 \in A_{ij}$

If $a_1 \notin A_{ij}$, then $\{a_3, a_4\}$ is optimal sol^n as $f(a_1) \leq f(a_3)$, then we can replace a_3 by a_1. Size remains same.

$\therefore \{a_1, a_2\}$.

Eg Design an algo. to solve A.S.P.
Write recursive algo.

R.A.S.P(s , f , k , n)

- s: Start time
- f: finish time
- k: Activity after this finishes
- n: Activity finishes before this start

```
1.  min_element = a_k ; i=k+1
2.  min = f(a_k)
5.  while ( f(a_k) ≤ s(a_i) and i≤n )
6.      if min > f(a_i)
7.          min = f(a_i)
8.          min_element = a_i
9.      i = i+1
10. RASP ( s, f, min_element , n)
11. Result . append (E) ;
```

```
3.  if  k == n
4.      return a_k
```

```
*   RASP (s, f, k, n);        k = 0
1.  m = k+1
2.  while m ≤ n and s[m] < f[k]
3.          m = m+1
4.  if m ≤ n
5.      return {a_m} ∪ RASP(s, f, m, n)
6.  else
7.      return φ
```

$$T(n) = \Theta(n)$$

* IASP (S, f) :

```
A = {a1}
n = S.length
for i = 1 to n
    K = 1
    for m = 2 to n
        if S[m] ≥ f[K]
            A = A ∪ {am}
            K = m
return A
```

*

a	b	c	d	e	f
45K	13K	12K	16K	9K	5K

Total = 100K char

Task: No. of bits to be transfered should be minimum.

⇒ Code:

a	b	c	d	e	f
000	001	010	011	100	101

- Total bits to be transfered = 3 × 100K
  ⇒ = 300,000 bits
  (Max-Case)

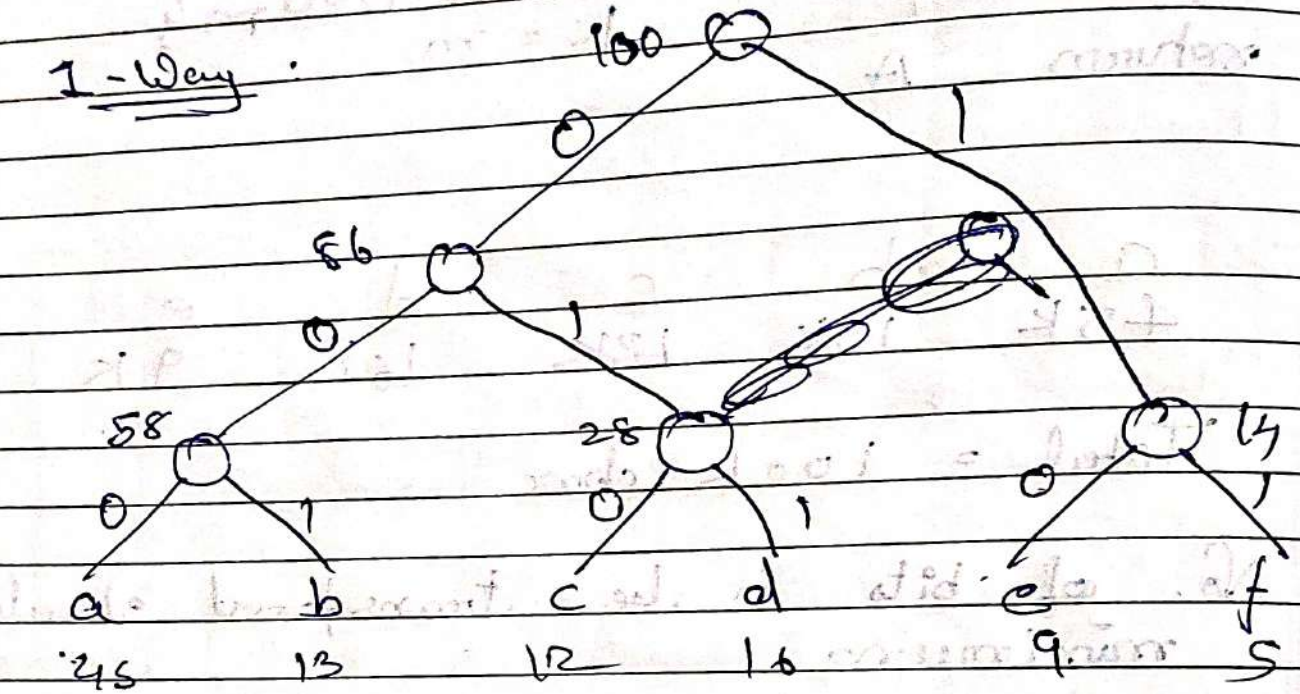
Task: Assign codeword to letters such that total bits are less than 300K.

a	b	c	d	e	f

→ Automation : Code generator

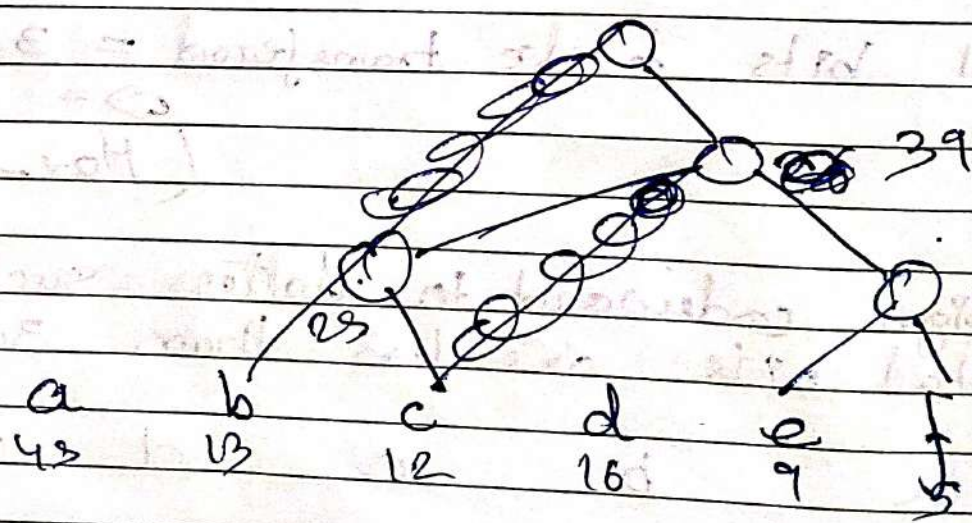
When do we have collisions
↳ Prefix free codewords
↳ No codeword should be a prefix of other codeword.

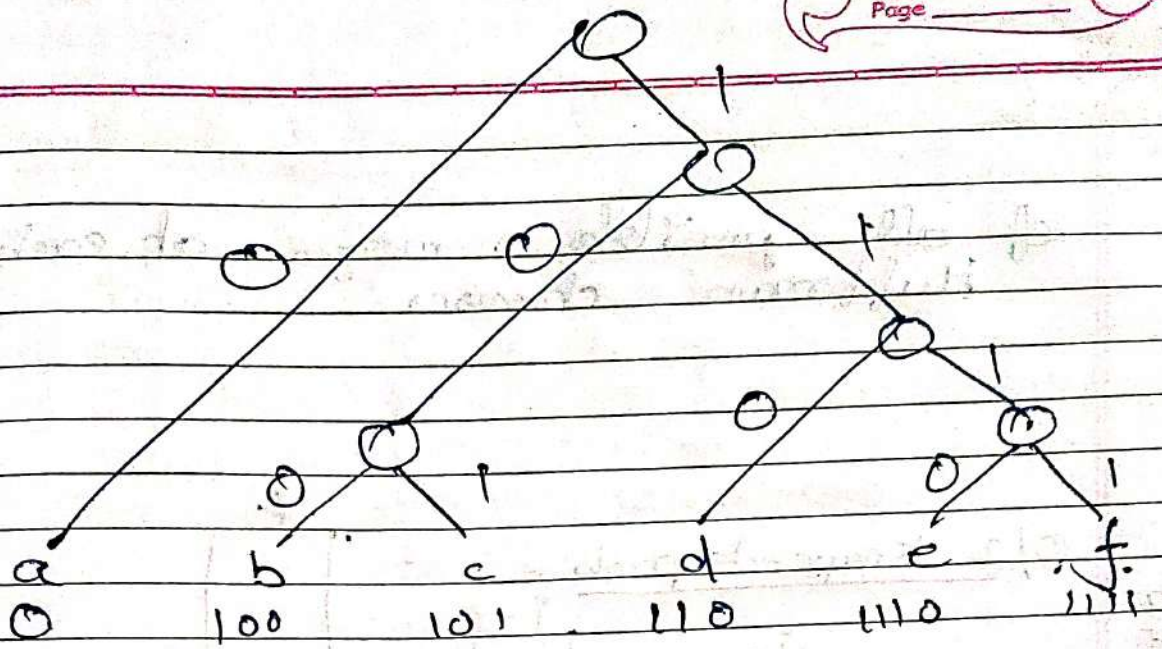
→ 1-Way :



000 001 010 011 10 11

→ Better Way :





① Find two lowest freq char.
Resolve them.
(Implement using min heap)

② Create a node = A.freq + B.freq = C.freq

✱ Huffman (C) :

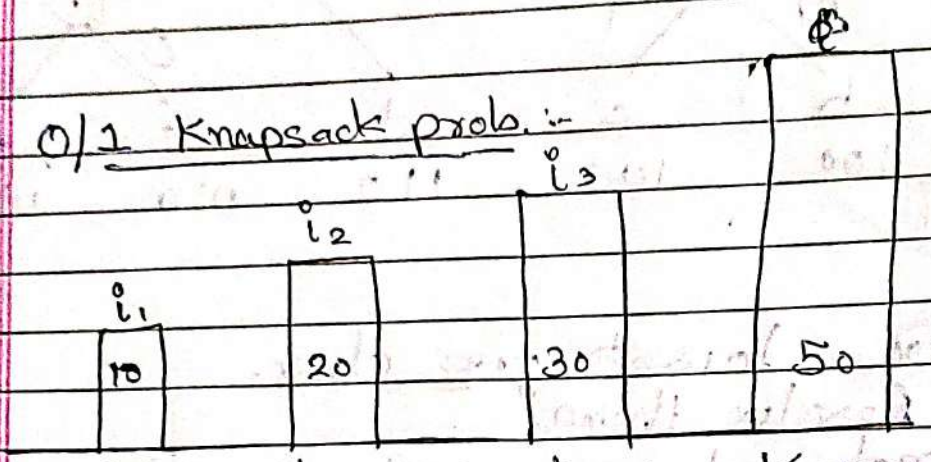
```
n = |C|
Sort (C)                                  heap
Θ(n)  Q = C  // Initialize Q with characters in C
Θ(n)  for i = 1 to n-1                                  Θ(nlogn)
          allocate a new node z                          ↙
          z.left  = x = Extract-min(Q)  ←
          z.right = y = Extract-min(Q)
          z.freq = x.freq + y.freq
          Insert (Q, z)  ←  Θ(nlogn)
      return Extract-min(Q)
```

$$T(n) = \Theta(n \log n)$$

of all possible merges at each step, Huffman chooses

0/1 Knapsack prob.:-

Eg:



Objective: To fill the knapsack & earn max. point
Splitting is not allowed.

→

Possible Ways:	Profit ($)
10	60
20	100
30	120
10 20	160
10 30	180
20 30	220

→ Greedy approach give (10, 20) as answer which is wrong. So, greedy cannot be applied.

Eg

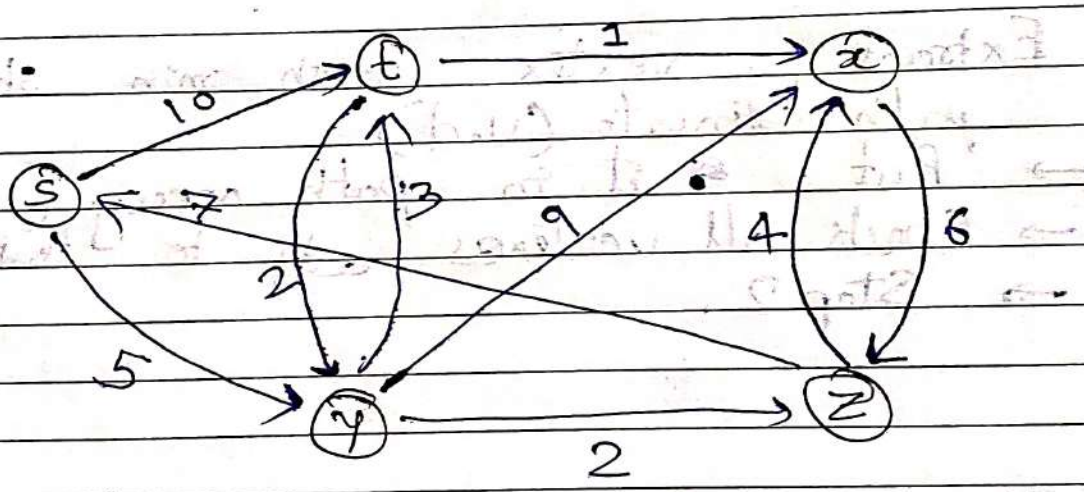
Fractional Knapsack : Splitting allowed.

Price per unit weight

i_1	i_2	i_3
6	5	4
$6 \times 10 = 60\$$	$5 \times 20 = 100\$$	$4 \times 20 = 80\$$

Total = 240 $

Eg



↳ Find shortest path from s to all other nodes

Ans Given : E : Set of edges
V : Set of vertices
s : Source vertex

$$s \xrightarrow{p} x = \min_{p \in V} \left(s \xrightarrow{p_1} p + p \xrightarrow{p_2} x \right)$$

Date
Page

In

① Initialize:

$v.d = \infty$
$v.\pi = Nil$
$s.d = 0$

	v.d	v.π
s	0	Nil
t	∞	Nil
x	∞	Nil
y	∞	Nil
z	∞	Nil

② If $y.d > x.d + w(x,y)$

$y.d = x.d + w(x,y)$
$y.\pi = x$

③ Extract a vertex with min. shortest path estimate (v.d)
→ Put it in path array (K)
→ Check all vertexes adj. to that vertex.
→ Step 2.

→ Store v.d in Min-heap

☆ Pseudocode :

① Initialize Vertices

② $u \xrightarrow{3} v$

If $v.d > u.d + w(u,v)$

then $v.d = u.d + w(u,v)$
$v.\pi = u$

(3) Extract a node from min-heap

(4) Evaluate edges using step 2

1. $s.d = 0$ and $s.\pi = Nil$
2. for $i = 2$ to $V.size()$
   $V[i].d = \infty$
   $V[i].\pi = Nil$
3. for $i = 1$ to $V.size()$
   if ($V[i] == s$)
   continue
   else
   if ($V[i].d \geq$

Initialize-Single-Source (G, s):

for each vertex $v \in G.V$ $\Theta(V)$
    $v.d = \infty$
    $v.\pi = Nil$
$s.d = 0$

Relax (u, v, w):

if $v.d > u.d + w(u,v)$
then $v.d = u.d + w(u,v)$
    $v.\pi = u$

Dijkstra (G, s):

$K = \phi$ // Result path
$Q = G.V$ // Store vertices in min-heap
while $Q \neq \phi$ $\rightsquigarrow \Theta(V)$
    $u = Extract\text{-}min(Q) \rightsquigarrow \Theta(V \log V)$
    $K = K \cup \{u\} \rightsquigarrow \Theta(1)$
    for each vertex $v \in G.adj[u]$ $\Theta(E \log V)$
        Relax (u, v, w)

$T(n) = \Theta(V \log V) + \Theta(E \log V)$

$= \boxed{\Theta((V+E) \log V)}$

s, v

Print-path (G, s, v);

dest = K.Pop()
while
dest = v
while v ≠ s

if v == s
| print s

else if v.π = Nil
| print "No path from s to v"

else
| print_path (G, s, v.π)
print v

$\Theta(V)$

Note: Instead of min-priority queue, if we solved using array what will be time complexity

NP - Completeness

→ Time Complexity

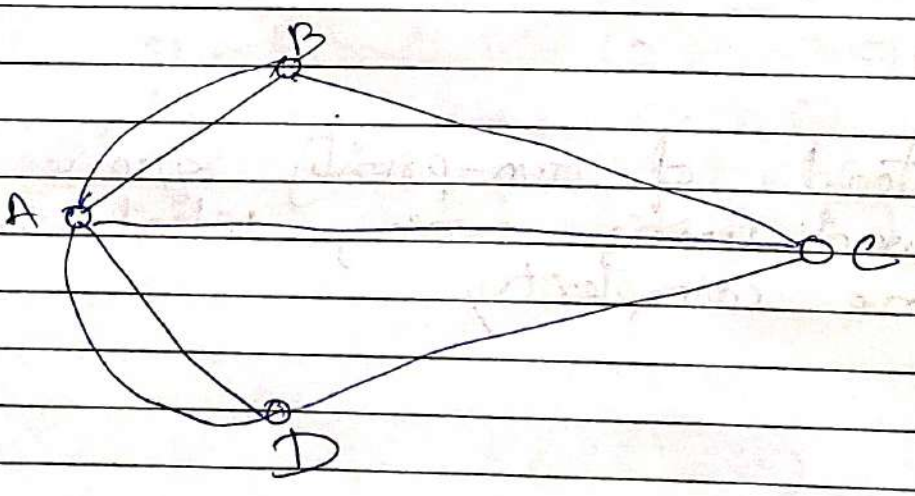
↳ $O(n^c)$ where c = constant

↳ If we solve any of NP - complete problem, we will be able to solve all NP - complete problems.

↳ Pairs of problems:
① Shortest path problem - Polynomial time algo
② Longest path problem - NP complete

Euler Tour ☆

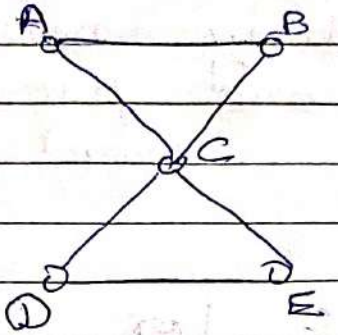
Given a connected, undirected graph, $G = (V, E)$ is a cycle that traverse each edge of a exactly one and it can visit vertices more than one.



Hamiltonian Cycle ☆

A H.C. of a connected undirected graph $G = (V, E)$ is a cycle that

Start and ends at same vertex and visits each vertex exactly one.



Note:- → A graph has Euler Tour if and only if degree of every vertex is even.

→ Hamiltonian cycle is NP Complete.

* Class P :- If a polynomial time algo. exists to solve a problem then the problem is in class P.

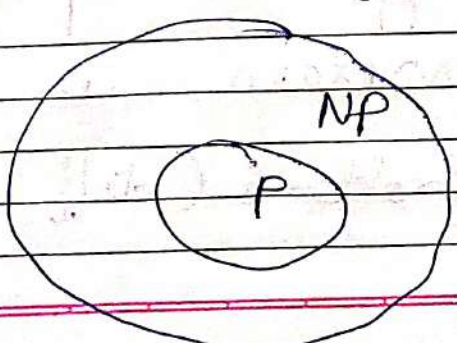
Eg, sorting
searching

* Class NP :- Consists of problems that are "verifiable" in polynomial time, given the sol^n.

Note:

P
↓
Solve.

NP
↓
Verify the correctness of sol^n (Certificate)



3^3 31
27

Date
Page

All problems in class P are in NP.
Is all problems in class NP also in P?
↳ Nobody knows
↳ No one proved that answer can/ cannot exits

* NP-Hard :- For any NP-Hard Problem if there is a polynomial time algorithm to solve a problem, then there is a way to convert every problem in NP into this problem in such a way to solve every problem in NP in polynomial time.

* Class NP Complete : (NPC)

↳ Two conditions
① Problem must be in class NP
② Problem is in class NP-Hard

te: $P \wedge NPC = \phi$

Three Key Concepts to show that problem is NPC.

) Decision Problems (diff - from optimization problems)

② Reductions

③ First NP Complete Problem

* Decision Problem :-

Optimisation Prob.	Decision Prob.
O/p = Path	O/p = Yes or No

↳ We can cast a given optimisation problem as a related decision problem.
↳ by imposing a bound on value to be optimised.

K = 4 ⟶

Shortest path

K = 8 Yes
K = 7 Yes
⋮
K = 4 Yes ✓
K = 3 No

Example: Cast an optimization problem to D.P.

O.P. : Given a directed graph G and vertices u & v, find shortest path from u to v.

D.P. : Given a directed graph G & vertices u & v and a positive integer k, does a path exists from u to v where path weight is atmost k.

→ D.P. is 'easier' or atleast 'not harder' than the O.P.

O.P.	D.P.
Easy	Easy
Hard	Easy
	Hard